AIQE GITHUB CAPSTONE

Project Handbook & Long-Term Maintenance Guide

A layman-friendly guide to understanding, running, troubleshooting, and improving the Playwright + TypeScript framework

Repository	malaysahacreative-alt/aiqe-github-capstone
Project owner	Malay Saha
Primary technologies	Playwright Test, TypeScript, Node.js, GitHub REST API, GitHub Actions
Documentation date	15 October 2026
Purpose	Help a future reader rebuild their understanding after a long gap—even if they are not a developer.

Prepared from the public repository contents available on the documentation date. Where the code and README differ, this handbook calls out the discrepancy rather than assuming the intended behaviour.

Contents

Use the PDF bookmarks panel for quick navigation. The main sections below are also bookmarked.

Section	What it covers
1. Executive summary	What the project is and why it exists
2. Quick-start map	How to approach the project after a long gap
3. Architecture	How UI, API, integration tests, and shared code fit together
4. Repository structure	Purpose of folders and files
5. Technology explained	Playwright, TypeScript, Node.js, JSON, REST APIs, Git
6. Test scenarios	UI, API, and integrated workflows
7. Configuration and secrets	.env, env.ts, tokens, local vs CI configuration
8. How to run the project	Setup, commands, reports, evidence
9. GitHub Actions CI	Automated test execution on GitHub
10. AI in the project lifecycle	What AI does, how to use it safely, sample prompts
11. Troubleshooting playbook	Common failures, likely causes, diagnostic steps
12. Findings and improvement plan	Repository-specific gaps to review
13. Maintenance after two years	Re-onboarding, upgrades, change management
14. Glossary and references	Plain-English definitions and trusted links

1. Executive summary

This repository is an automated quality-engineering (QA) project. Its job is to check selected behaviours of GitHub.com and GitHub's REST API without a person having to repeat every check manually. It uses Playwright Test to run checks and TypeScript to describe the test logic.

Think of it as a digital inspector. The inspector can open a website, find visible controls, read page details, call an API endpoint, inspect the response, and compare the result with what the test expects.

What the project demonstrates

- UI automation: checks the visible GitHub website, such as a profile or repository page.
- API automation: sends requests to GitHub's REST API and checks status codes and returned data.
- Page Object Model (POM): keeps page-specific locators and actions in reusable classes.
- Service layer: centralises API requests in GitHubService rather than repeating request code in every test.
- Configuration and test data: separates environment settings and reusable values from test logic.
- Integrated validation: uses API information to navigate to a repository page and verifies related UI details.
- CI/CD: GitHub Actions can run the test suite when code is pushed or a pull request is opened.
- Responsible AI: AI can assist with design, code drafting, explanations, and debugging, but a human must verify the result.

What it does not automatically mean

A test passing is evidence that the specific assertion passed under the conditions of that run. It is not proof that all GitHub functionality works, that every browser is supported, or that the framework is production-ready. Coverage depends on the test cases, data, environment, permissions, and assertions.

Important distinction: The inspected code shows AI being documented as a development aid (for example, an AI usage log). The reviewed files do not show an AI model being called at runtime to generate or execute tests automatically. AI-assisted development and an AI-powered running application are different things.

2. Quick-start map: returning after a long gap

If you open this project after months or years, do not start by editing code. First establish what the repository contains, how to install it, whether secrets are configured, and whether the current baseline runs.

Order	Action	Why it matters
1	Open the repository README and this handbook.	Understand intended scope before changing files.
2	Check the current branch, latest commit, and recent workflow runs.	Learn what changed and whether CI is already failing.
3	Install a supported Node.js LTS version.	The package manager and Playwright depend on Node.js.
4	Run npm ci.	Recreate dependencies from package-lock.json reproducibly.
5	Create a local .env from .env.example.	Provide the configuration required by tests without committing secrets.
6	Inspect config/env.ts and playwright.config.ts.	Confirm how environment variables and browser settings are loaded.
7	Run one small UI test, then one API test, then the full suite.	Isolate problems instead of debugging everything at once.
8	Open the HTML report and inspect failure evidence.	Separate assertion failures from setup, authentication, or network failures.

Order	Action	Why it matters
9	Review GitHub Actions logs and secrets configuration.	Local success does not guarantee CI has the same environment.
10	Only then change code—and record the reason.	Preserve a known baseline and make regressions easier to identify.

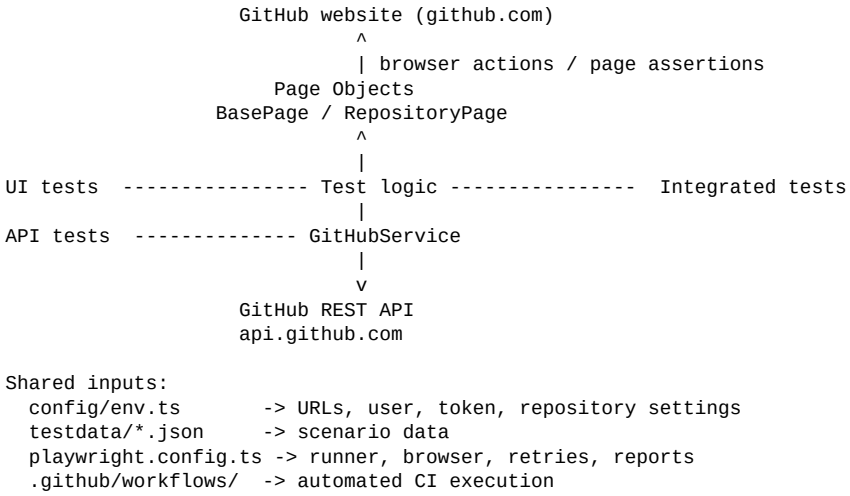
A useful first terminal session

```
cd aiqe-github-capstone
git status
git log -5 --oneline
node --version
npm --version
npm ci
npx playwright --version
npx playwright test --list
```

If npm ci fails, read the first meaningful error. It can indicate a Node.js compatibility issue, a lockfile mismatch, a network issue, or an installation problem. Do not immediately delete the lockfile; it records the dependency versions used to reproduce the environment.

3. Architecture: how the pieces work together

The project has three kinds of test journey: UI-only, API-only, and integrated. They share configuration and helper code.



A typical UI test

- The test chooses a scenario and opens a browser page.
- A page object uses a locator to identify an element such as a search box or repository name.
- Playwright performs an action or reads the page.
- An assertion checks the observed result against the expected result.
- If an assertion fails, the report and configured screenshot/trace can help diagnose why.

A typical API test

- The test creates or receives a Playwright API request context.
- GitHubService builds a request URL and sends HTTP headers, including the token for authenticated endpoints.
- GitHub responds with an HTTP status and a response body, usually JSON.
- The test checks the status and important fields, such as repository name and owner.

An integrated test

- The test requests repository details from the API.
- It reads the repository's HTML URL and name from the API response.
- It opens that URL in a browser.
- The repository page object verifies that the visible repository name and URL correspond to the API data.

The integrated pattern is useful because it compares two views of related information. The API is good at returning structured data; the browser is good at verifying what a user sees. The API and UI checks can fail for different reasons, so diagnostic evidence matters.

4. Repository structure: what each folder is for

Path	Plain-English purpose	What to look at
.github/workflows/playwright.yml	Instructions GitHub Actions follows to run tests in the cloud.	Triggers, Node version, install steps, test command, artifacts.
config/env.ts	Reads and validates environment variables.	Required values, default URLs, missing-variable errors.
src/pages/BasePage.ts	Common browser-page operations shared by page objects.	Navigation, title/URL reading, screenshot helper.
src/pages/GitHubHomePage.ts	Actions specific to GitHub's home page.	Search field locator and repository search action.
src/pages/ProfilePage.ts	Actions and checks for a user profile page.	Profile navigation, URL, repositories tab.
src/pages/RepositoryPage.ts	Actions and checks for a repository page.	Repository name, owner, Code tab, URL.
src/services/GitHubService.ts	Central API communication helper.	Authorization header, endpoint URLs, response calls.
src/utils/DataUtil.ts	Small reusable data-generation functions.	Timestamp and generated repository-name helpers.
testdata/uiData.json	Values used by UI scenarios.	Repository owner/name and profile name.
testdata/apiData.json	Values used by repository API scenario.	API owner and repository name.
tests/ui/	Browser-focused tests.	Expected visible behaviour and page-object usage.
tests/api/	API-focused tests.	Endpoint calls, status codes, response assertions.
tests/integrated/	Tests combining API and browser checks.	How API output is reused in the UI journey.
.env.example	Safe template for local configuration.	Variable names and placeholder values only.
.env (local only)	Real local settings, potentially including a token.	Never commit or paste real secret values.
.gitignore	Files Git should not track.	Secrets, dependencies, reports, auth state, screenshots.
package.json	Project metadata and npm scripts/dependencies.	Scripts and direct dependencies.
package-lock.json	Exact dependency resolution record.	Reproducible installation; update alongside package changes.
playwright.config.ts	Test runner settings.	Test directory, browser, headless mode, retries, report, trace.
README.md	Project's quick overview and run instructions.	Compare documented commands with actual configuration.
AI_Usage_Log.md	Record of AI assistance in the project.	Expand it with prompts, changes, review, and validation evidence.

How to read the source in a sensible order

Start with package.json and playwright.config.ts to understand how tests run. Then read config/env.ts to understand required settings. Next read one test file, then the page object or service it calls. Read test data last, because its meaning becomes clearer once you see which test imports it.

A note about names

The repository has a testdata folder (not test-data), and the inspected configuration file is config/env.ts. Use the exact case-sensitive path shown by GitHub when opening files. File and folder names on some systems are case-sensitive.

5. Technology explained without assuming coding experience

Playwright Test

Playwright is a browser automation and testing framework. Playwright Test is its test runner: it discovers tests, manages browser contexts, runs assertions, retries where configured, and creates reports. It can also send HTTP API requests.

TypeScript

TypeScript is JavaScript with a type system. Types help describe the kind of value expected—for example, a username as text or a page object as a Playwright Page. TypeScript does not automatically guarantee that every runtime value is correct; tests and validation are still necessary.

Node.js and npm

Node.js runs JavaScript/TypeScript tooling outside the browser. npm installs packages and runs named scripts. package.json lists project dependencies and scripts; package-lock.json locks down the resolved dependency tree.

JSON test data

JSON is a structured text format for storing data. A test can import values from a JSON file rather than hardcoding them repeatedly. JSON is data, not executable test logic.

```
{
  "owner": "malaysahacreative-alt",
  "repository": "aiqe-github-capstone"
}
```

In this example, owner identifies the GitHub account or organisation and repository identifies the repository name. The API test imports these values and passes them to GitHubService.

REST API and HTTP status codes

Term / code	Plain-English meaning
Endpoint	A URL representing a resource or operation, such as /repos/{owner}/{repo}.
GET	Ask a server to return information.
200 OK	The request succeeded.
401 Unauthorized	Credentials are missing, invalid, or not accepted.
403 Forbidden	The request is understood but not allowed; rate limits can also produce this.
404 Not Found	The resource may not exist or the caller may not be allowed to see it.
429 Too Many Requests	The service is limiting request frequency (service-specific behaviour).
JSON response	Structured data returned by the API.
Bearer token	A credential sent in an Authorization header; treat it as a password.

Locators and assertions

A locator is a reusable description of how to find a page element. Examples include locating a button by its accessible name or an element by a test ID. An assertion is the check that decides whether the observed result matches the expectation.

```
const searchInput = page.getByRole('combobox', { name: /search/i });
await searchInput.fill('playwright');
await searchInput.press('Enter');

await expect(page).toHaveTitle(/GitHub/i);
```

Prefer locators that describe the element's role and accessible name when possible. Test IDs can be appropriate when the application provides a stable test hook. Avoid selecting elements by fragile CSS structure unless there is a good reason.

Page Object Model (POM)

POM means keeping page-specific details in a class. A test can say “validate repository name” instead of repeating the locator and assertion in many files. This reduces duplication, but a page object should not hide every detail or become a huge class.

Git, GitHub, and CI

- Git tracks changes in files and allows developers to compare versions.
- GitHub hosts repositories and provides collaboration features.
- Continuous integration (CI) runs checks automatically when changes are pushed or proposed in a pull request.
- An artifact is a file produced by a workflow, such as an HTML test report, that can be downloaded after the run.

6. Test scenarios explained

6.1 UI: GitHub home page

The README describes a home-page validation that checks the page title and landing page accessibility. GitHubHomePage.ts contains a search locator and an action to search for a repository. A test should verify the actual intended outcome after the search—not just that the input accepted text.

6.2 UI: public user profile

ProfilePage.ts navigates to a profile path and checks the profile URL and visibility of a repositories link. The profile name is supplied as an argument. The test data currently lists torvalds as a profile value.

6.3 UI: repository page

RepositoryPage.ts defines locators for the repository name, owner, and Code tab. It offers methods to validate the name, owner, tab visibility, and URL. The owner locator currently refers directly to malaysahacreative-alt, so the method is not fully generic even though validateRepositoryOwner accepts an expectedOwner argument.

6.4 API: authenticated user

The README describes GET /user. This endpoint requires authentication. The test should validate a successful status, the login and numeric user ID, and a suitable JSON content type. If the token is invalid or unavailable, the test may fail before any response-body assertion.

6.5 API: repository details

tests/api/repositoryApi.spec.ts imports testdata/apiData.json, calls GitHubService.getRepository(owner, repository), expects status 200, then checks name, owner login, full name, default branch, and HTML URL.

```
const response = await githubService.getRepository(
  apiData.owner,
  apiData.repository
);

expect(response.status()).toBe(200);

const responseBody = await response.json();
expect(responseBody.name).toBe(apiData.repository);
expect(responseBody.owner.login).toBe(apiData.owner);
```

This checks more than whether the endpoint responds. It checks that the response belongs to the expected repository.

6.6 API: list user repositories

The README describes GET /user/repos and checks the status, that the response is an array, the repository count, and whether the capstone repository is present. Remember that the result depends on the authenticated user's permissions and any pagination settings.

6.7 Integrated API + UI test

tests/integrated/repositoryApiUiValidation.spec.ts requests the capstone repository using the API, expects status 200, reads the JSON, opens repository.html_url, and validates the repository name and URL through RepositoryPage. This is a useful end-to-end cross-check.

Test-data maintenance: If the target repository, owner, or test profile changes, update the corresponding data and any hardcoded locators/URLs consistently. Otherwise, one test may point at a new repository while another still expects the old owner.

7. Configuration and secrets

What config/env.ts does

The current config/env.ts imports dotenv/config, defines a helper called requiredEnv, and exports an env object. The helper trims a variable and throws a clear error when it is missing. UI and API base URLs have default values; user ID, token, and test repository are required.

```
GITHUB_UI_BASE_URL=https://github.com
GITHUB_API_BASE_URL=https://api.github.com
GITHUB_USER_ID=your_github_username
GITHUB_TOKEN=replace_with_your_token
GITHUB_TEST_REPO=aiqe-github-capstone
```

These are example values only. Create a local .env file from .env.example and replace placeholders with values appropriate for your account. Never include a real token in documentation, screenshots, chat messages, or a public repository.

Token safety rules

- Use a fine-grained token scoped to the required repository and minimum necessary permissions.
- Keep .env local and confirm it is ignored by Git before committing.
- Use GitHub Actions repository or environment secrets for CI credentials; do not put the token directly in a YAML workflow.
- If a token is exposed, revoke it and create a replacement. Deleting the visible text alone may not remove it from Git history.
- Do not paste secrets into AI assistants. Replace them with placeholders such as GITHUB_TOKEN=REDACTED.
- Review permissions and expiration periodically; authentication can fail when tokens expire or scopes change.

Local configuration versus CI configuration

On your Mac, dotenv/config can load a local .env file. GitHub Actions runners do not automatically receive that local file. The workflow must explicitly provide the required values through environment variables and GitHub Secrets. A repository secret should be referenced by name in the workflow; the secret value should never be printed.

```
# Illustrative pattern only; add the appropriate repository secret in GitHub Settings.
- name: Run Playwright tests
  run: npx playwright test
  env:
    GITHUB_USER_ID: ${{ secrets.GITHUB_USER_ID }}
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
    GITHUB_TEST_REPO: aiqe-github-capstone
    GITHUB_UI_BASE_URL: https://github.com
    GITHUB_API_BASE_URL: https://api.github.com
```

The example is a pattern to review, not a claim that the current workflow already injects these variables. Check actual repository settings and workflow runs before relying on CI.

Browser storage state

playwright.config.ts references storageState: 'playwright/.auth/user.json'. This file represents saved browser authentication state if it has been created. It is ignored by the repository's .gitignore. If a test genuinely needs a signed-in browser session, define a safe setup process and keep the state file out of source control. API-token authentication and browser-login state are separate mechanisms.

8. How to install and run the project

Prerequisites

- A supported Node.js LTS release and npm.
- Git, to clone and inspect the repository.
- A code editor such as VS Code (optional but helpful).
- Network access to GitHub and GitHub API.
- A valid GitHub token only for tests that call authenticated endpoints.

Step 1: get the source

```
git clone https://github.com/malaysahacreative-alt/aiqe-github-capstone.git
cd aiqe-github-capstone
```

Step 2: install packages and browsers

```
node --version
npm --version
npm ci
npx playwright install
```

On Linux CI, Playwright's browser dependencies may also be required; the repository workflow uses npx playwright install --with-deps.

Step 3: create local configuration

```
cp .env.example .env
# Edit .env in a text editor and replace placeholder values.
# Never paste the actual token into a command that may be saved in shell history.
```

The .env.example file is a template. The current config/env.ts requires GITHUB_USER_ID, GITHUB_TOKEN, and GITHUB_TEST_REPO. If these are absent, the module should throw an explicit missing-variable error.

Step 4: list and run tests

```
npx playwright test --list
npx playwright test
npx playwright test tests/api/repositoryApi.spec.ts
npx playwright test tests/integrated/repositoryApiUiValidation.spec.ts
npx playwright show-report
```

Use the paths that exist in the current branch. If you run a specific test and Playwright says no tests were found, check the exact file path and spelling.

Npm scripts

The README advertises commands such as npm run test:ui, npm run test:api, and npm run test:integrated. However, the inspected package.json currently has an empty scripts object. Unless those scripts are added, these npm run commands will fail. Until the mismatch is corrected, use the npx playwright test commands above.

```
{
  "scripts": {
    "test": "playwright test",
    "test:ui": "playwright test tests/ui",
    "test:api": "playwright test tests/api",
    "test:integrated": "playwright test tests/integrated",
    "report": "playwright show-report"
  }
}
```

This is a suggested script block, not a statement that it is already present. Merge it carefully into package.json and commit the corresponding change.

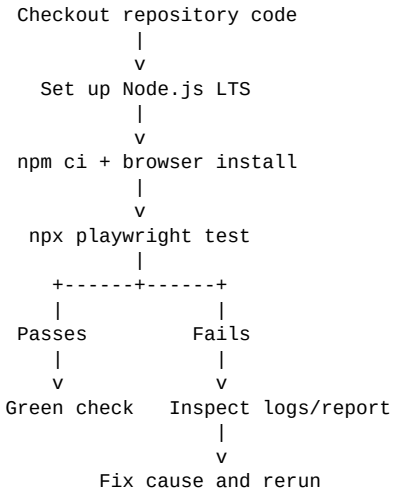
Reports and evidence

The configuration uses the HTML reporter, screenshots on failure, and trace collection on the first retry. Open the HTML report after a run. For a failure, inspect the assertion message, URL, screenshot, trace, response status, and workflow logs. Do not upload reports publicly if they contain private page content or sensitive data.

9. GitHub Actions: what happens automatically

The current workflow at .github/workflows/playwright.yml is named Playwright Tests. It is configured to run on pushes and pull requests targeting main or master. It checks out the code, sets up Node.js LTS, runs npm ci, installs Playwright browsers with OS dependencies, runs npx playwright test, and uploads the Playwright report as an artifact for up to 30 days.

```
Developer pushes code / opens pull request
                 |
                 v
         GitHub Actions starts
                 |
                 v
```



CI configuration items to verify

- Provide required environment variables through GitHub Secrets or non-sensitive workflow values.
- Confirm the workflow's Node.js version is compatible with the project's Playwright and dotenv versions.
- Ensure the test command matches the intended suite and that the HTML report path is correct.
- Check that failures upload useful evidence even when tests fail; the current workflow uses an if condition intended to upload unless cancelled.
- Use least-privilege token permissions and avoid printing credentials in logs.

A workflow can start correctly and still fail because a required token is missing, an authentication state file does not exist, a browser dependency is unavailable, or a test assertion is outdated.

10. AI in the project lifecycle

What AI can help with

Activity	How AI can help	Human verification needed
Requirement analysis	Turn a plain-English requirement into positive, negative, and boundary scenarios.	Confirm business intent and acceptance criteria.
Framework design	Suggest folders, POM classes, service layers, and naming conventions.	Ensure design fits the existing repository; avoid unnecessary rewrites.
Test drafting	Generate an initial Playwright test or TypeScript helper.	Run it, check locators, assertions, types, and maintainability.
Locator assistance	Explain DOM snippets and suggest accessible locators.	Verify locator uniqueness and stability on the real page.
Failure diagnosis	Explain stack traces and rank possible root causes.	Use actual logs and reproduce before changing code.
Test-data design	Suggest representative and edge-case JSON data.	Avoid production personal data and validate the schema.
Documentation	Explain code and keep onboarding notes readable.	Compare explanations with the current source.
Code review	Spot duplicated code, weak assertions, hardcoded URLs, or missing error handling.	Run lint/type checks/tests and review security implications.
CI improvement	Draft workflow steps and artifact handling.	Check secret injection, permissions, branch triggers, and run results.
Test maintenance	Suggest updates when the application UI or API changes.	Never accept an auto-generated locator or assertion without checking it.

AI-assisted development is not the same as runtime AI

In the reviewed project files, AI is mentioned in AI_Usage_Log.md and the README's responsible-AI section. The log currently records framework design assistance such as folder structure creation and POM design. The files reviewed do not show a runtime call to a large language model (LLM), an AI agent, or an AI API. The project can be AI-assisted in how it was built without being an AI system during test execution.

A safe AI workflow

```
1. Define the requirement and acceptance criteria.
2. Ask AI for a proposed design or small code change.
3. Review every changed file; compare it with the existing architecture.
4. Check for secrets, unsafe permissions, invented APIs, and brittle locators.
5. Run the smallest relevant test.
6. Run the full suite and inspect the report.
7. Record what AI suggested, what was accepted/rejected, and the test evidence.
8. Commit only verified changes.
```

Example prompt: explain existing code

```
You are reviewing an existing Playwright + TypeScript repository.
Explain this file for a beginner:
- its purpose
- each import and important statement
- inputs and outputs
- files it depends on
- what could fail and how to diagnose it
Do not invent behaviour that is not visible in the code.
Do not rewrite the file. List uncertainties separately.
```

Example prompt: generate a test safely

```
Create a Playwright TypeScript test for this requirement:
[PASTE REQUIREMENT]

Before writing code, list the assumptions and acceptance criteria.
Use the existing Page Object Model and API service layer.
Reuse existing test data when appropriate.
Do not hardcode secrets or add new dependencies without explaining why.
Use stable, accessible locators and meaningful assertions.
Return the proposed test, files to change, risks, and exact commands to validate it.
Do not claim it passes until the test has actually been run.
```

Example prompt: diagnose a failure

```
Analyse this Playwright failure using only the supplied evidence.
1. Identify the first meaningful error.
2. Separate symptoms from likely root causes.
3. Rank causes and explain how to test each one.
4. Suggest the smallest safe code/config change.
5. Give commands to reproduce and verify the fix.
Do not ask me to paste tokens, cookies, or .env contents.
Failure output:
[PASTE REDACTED ERROR AND RELEVANT CODE]
```

How to document AI usage

Expand AI_Usage_Log.md beyond a brief framework-design note. For each meaningful use, record the date, task, prompt summary (not secrets), files affected, accepted and rejected suggestions, human reviewer, validation commands, result, and known limitations. This creates a traceable record for future maintainers.

```
## Example entry
- Date: YYYY-MM-DD
- Goal: Add repository API validation
- AI assistance: Suggested assertion list and test structure
- Files changed: tests/api/repositoryApi.spec.ts
- Human review: Confirmed endpoint, response fields, and expected values
- Validation: npx playwright test tests/api/repositoryApi.spec.ts
- Result: Record actual pass/fail and report location
- Follow-up: Record any unresolved issue
```

Potential next step: AI at runtime

If the long-term goal is an AI-powered test assistant, that is a separate feature and should be designed explicitly. Possible capabilities include turning approved requirements into draft test cases, summarising failures, or suggesting locator repairs. Add controls for human approval, sandbox execution, token/cost limits, audit logs, and safe handling of repository data. Never let an LLM silently modify production code or execute destructive actions merely because it generated them.

11. Troubleshooting playbook

Symptom	Likely causes	First checks
Missing required environment variable	.env not created, typo in variable name, CI secret not mapped.	Check variable names only; do not print values. Review config/env.ts.
401 from GitHub API	Token invalid, expired, malformed, or not accepted.	Check token status, Authorization header format, secret mapping.
403 from GitHub API	Insufficient permissions, rate limit, organisation policy.	Inspect response body and rate-limit headers; verify least required scope.
404 for repository	Wrong owner/name, repository moved/deleted, or access restrictions.	Check apiData.json, repository URL, and authenticated account access.
UI locator times out	GitHub UI changed, wrong page, locator not unique, page not loaded.	Check screenshot/trace and actual page URL; inspect locator and accessibility name.
Strict-mode violation	A locator matches multiple elements when one was expected.	Make the locator more specific using role/name, container, or test ID.
Navigation fails	Network problem, redirect, invalid URL, rate limit, browser issue.	Check URL, response, timeout, network, and trace.
Browser launch fails	Browser binaries missing or incompatible OS/runtime.	Run npx playwright install; inspect supported OS and Node version.
npm run test:api fails	Script missing from package.json.	Use npx playwright test tests/api or add the documented script.
CI fails but local run passes	CI lacks .env/secrets/auth state, OS differs, timing differs.	Compare workflow logs, environment variable names, browser versions, and artifacts.
Screenshot path error	Output folder absent or path not writable.	Ensure the screenshots directory exists or create it safely before writing.
Report not available	Tests did not start, reporter path differs, artifact not uploaded.	Check workflow step status and the configured report output path.

A disciplined debugging sequence

- Copy the exact first meaningful error and note which test and line failed.
- Determine whether the failure is setup, authentication, network, locator, assertion, or CI-only.
- Reproduce only the failing test with a line reporter.
- Inspect the API status/body or UI screenshot/trace, while redacting secrets.
- Change one likely cause at a time.
- Rerun the failing test, then the related group, then the full suite.
- Record the root cause and evidence so the next person does not repeat the investigation.

```
npx playwright test tests/api/repositoryApi.spec.ts --reporter=line
npx playwright test tests/integrated/repositoryApiUiValidation.spec.ts --reporter=line
npx playwright show-report
```

Never share: GitHub tokens, cookies, browser storage-state JSON, or an unredacted .env file. Error logs should be reviewed for secrets before being sent to an AI assistant or added to an issue.

12. Repository-specific findings and improvement plan

The following observations are based on the current public source files inspected for this handbook. They are a maintenance review, not a claim that every item causes an active failure. Verify each change with a test run before merging.

Priority	Finding	Why it matters / suggested action
High	package.json has an empty scripts object, while README documents npm run test:ui/test:api/test:integrated.	Add the scripts or correct the README so the documented commands actually work.
High	playwright.config.ts has dotenv loading commented out, while config/env.ts imports dotenv/config.	The env module should load dotenv when imported, but make this dependency clear and test it. Avoid confusing duplicate loading strategies.
High	CI workflow runs npx playwright test but does not visibly map GITHUB_TOKEN/GITHUB_USER_ID/GITHUB_TEST_REPO into the step environment.	Authenticated API tests may fail in CI unless these are supplied another way. Configure GitHub Secrets and explicit env mapping.
High	playwright.config.ts references playwright/.auth/user.json as storageState.	Ensure the file is created by a setup flow when needed; do not commit it. If no browser login is required, consider removing this global dependency.
Medium	UI page objects contain hardcoded GitHub URLs and a hardcoded repository owner locator.	Use env.uiBaseUrl and scenario data where practical, so tests are easier to reuse and less brittle.
Medium	RepositoryPage.validateRepositoryOwner(expectedOwner) uses a locator preconfigured to match malaysahacreative-alt.	Make the locator strategy compatible with expectedOwner, or explicitly name the method as a fixed-owner check.
Medium	BasePage.takeScreenshot writes to screenshots/{name}.png.	Create the directory safely or use Playwright's configured test-results paths; verify it works from a clean checkout.
Medium	README's run commands and configuration should be reconciled with the actual scripts and CI behaviour.	Treat README as the user manual; keep it aligned with code after every structural change.
Medium	AI_Usage_Log.md currently contains only a short framework-design entry.	Record meaningful AI-assisted changes and the human validation evidence for auditability.
Medium	The workflow installs latest LTS Node and package.json uses version ranges for dependencies.	For repeatability, document supported Node versions and use lockfile-based npm ci; update dependencies deliberately.
Low	Repository name generation in DataUtil uses Date.now().	Good for many quick unique names but can collide if called multiple times within the same millisecond; add a random suffix if uniqueness becomes important.

Suggested order of improvements

- First: align package.json scripts, README instructions, and the actual workflow.
- Second: confirm local and CI environment-variable setup and token permissions.
- Third: decide whether storageState is needed for all tests or only a dedicated authenticated UI flow.
- Fourth: remove hardcoded URLs/owners where test data or configuration should drive behaviour.
- Fifth: improve error messages, screenshot folder handling, and test-data validation.
- Sixth: add more scenarios, reporting, multi-environment support, and any AI runtime features only when requirements justify them.

Avoid combining all changes in one large rewrite. Small pull requests with focused tests make it easier to identify which change introduced a regression.

13. Maintaining the project after two years

The first hour back

- Read this handbook and README; confirm the repository URL and default branch.
- Review recent commits, issues, pull requests, and GitHub Actions history.
- Check Node.js, npm, Playwright, and dependency versions.
- Review .env.example and the current secret names; do not expose secret values.
- Install from a clean checkout and run one UI, one API, and one integrated test.
- Compare current behaviour with the known limitations in Section 12.

Before upgrading Playwright

- Read the Playwright release notes and breaking changes for the version range being considered.
- Upgrade in a dedicated branch and keep package-lock.json consistent.
- Install the browser binaries matching the upgraded Playwright version.
- Run the full suite locally and in CI; inspect screenshots and traces for changed UI behaviour.
- Update this handbook and README with verified changes.

When GitHub changes its UI or API

Do not assume every failure is a bug in your test. GitHub may change accessible names, page structure, API response details, authentication requirements, or rate limits. Check the real page/response first, then update locators or assertions deliberately. Keep assertions strong enough to detect real regressions.

Suggested change record

Field	Example of what to record
Change ID	Issue or pull-request number
Reason	What requirement or failure motivated the change
Files changed	Exact paths
Risk	Which scenarios may be affected
Validation	Exact commands and environment
Result	Actual pass/fail summary and report link
Rollback	How to revert safely if the change breaks the suite
Documentation	Which handbook/README sections were updated

Quarterly maintenance checklist

- ■ Review dependency updates and security advisories.
- ■ Check whether GitHub token permissions and expiration remain appropriate.
- ■ Confirm GitHub Actions is green and artifacts are available.
- ■ Remove obsolete tests and update assertions when requirements change.
- ■ Check for accidental secrets or generated files in Git history and commits.
- ■ Review AI usage log and ensure generated code has recorded human validation.
- ■ Verify setup instructions from a clean environment rather than relying on an old laptop state.

14. Glossary

Term	Plain-English definition
API	A defined way for one software system to request data or actions from another.
Assertion	A check that compares what happened with what should happen.
Authentication	Proving who you are, often using a token or login session.
Authorization	What an authenticated user or token is allowed to do.
CI/CD	Automated processes for checking, building, and delivering software changes.
Environment variable	A named setting supplied outside source code, often used for URLs or credentials.
Fixture	A reusable resource Playwright provides to a test, such as a page or API request context.
Headless browser	A browser running without a visible window.
JSON	A text format for structured data such as names, URLs, and settings.
Locator	A Playwright object describing how to find an element on a page.
POM	Page Object Model: a design approach that puts page-specific actions and locators in classes.
Regression	A new problem introduced after a change to code, configuration, or dependencies.
Repository	A project location that stores files and their change history.
Retry	Another attempt to run a test after a failure.
Secret	Sensitive information such as a personal access token or password.
Service layer	Reusable code that centralises communication with an external service or API.
Trace	A diagnostic recording of a Playwright test run, useful for investigating failures.
Workflow artifact	A file saved from CI, such as a report, log, or test evidence.

15. Reference links

Project repository: https://github.com/malaysahacreative-alt/aiqe-github-capstone

Playwright documentation: https://playwright.dev/docs/intro

Playwright API testing: https://playwright.dev/docs/api-testing

Playwright configuration: https://playwright.dev/docs/test-configuration

GitHub Actions documentation: https://docs.github.com/actions

GitHub REST API: https://docs.github.com/rest

GitHub fine-grained personal access tokens: GitHub documentation on personal access tokens

Node.js download and release information: https://nodejs.org/en/about/previous-releases

Final operating principle

The most valuable framework is not the one with the most files or the cleverest code. It is the one a new maintainer can understand, run, diagnose, and safely change. Keep the source, test data, configuration, workflow, README, AI usage log, and this handbook aligned as the project evolves.

End of handbook.